

SMART STUDENT PERFORMANCE & ATTENDANCE MANAGER

Programming in Java — Evaluated Project

Student Name	SAMEER RAJBHAR
Registration Number	25BAI11033
Course / Branch	B.Tech CSE (AIML)
Project Type	Command-Line Java Application
Technology	Java 17+ / Java SE standard libraries
Storage	CSV file using java.nio.file

Prepared for course evaluation

1. Introduction

The Smart Student Performance & Attendance Manager is a command-line Java application designed to demonstrate practical use of Programming in Java concepts. It maintains student records, attendance information, marks, and academic reports. The system uses a modular package structure and standard Java libraries so that it can be compiled and executed directly from a terminal.

2. Problem Statement

Teachers and academic coordinators may need to repeatedly enter student information, calculate attendance, manage marks, and prepare summaries. A small modular command-line application can organize these tasks and reduce repeated manual calculations while demonstrating Java programming concepts.

3. Objectives

The objectives are to implement student CRUD operations, calculate attendance percentages, calculate marks averages and grades, generate basic reports, persist records in a CSV file, validate user input, and demonstrate modular Java design.

4. Functional Requirements

The system contains the following major functional modules:

- **Student Management:** add, view, search, update, and delete students.
- **Attendance Management:** record present/total classes and calculate attendance percentage.
- **Marks & Performance:** enter subject marks, calculate average and grade, and generate performance lists.
- **Reports:** show student counts, attendance alerts, top students, and students needing attention.

Additional functionality includes CSV persistence, validation, and error handling.

5. Non-Functional Requirements

- **Usability:** menu-driven commands and clear prompts.
- **Reliability:** invalid numeric input and common file errors are handled.
- **Maintainability:** model, service, storage, and utility packages separate responsibilities.
- **Resource efficiency:** in-memory collections and try-with-resources for file operations.
- **Portability:** no external libraries or database server are required; Java 17+ is sufficient.

6. System Architecture

The application follows a lightweight layered architecture. The Main class handles the command-line workflow. Service classes contain application logic. Model classes represent data. CsvStorage handles persistence. Utility classes handle validation and console presentation.

7. Design Diagrams

The repository includes architecture and UML design notes in **docs/architecture.md** and **docs/diagrams.md**. The main relationships are: Main uses StudentService, AttendanceService, PerformanceService, CsvStorage, ConsoleUI, and InputValidator; service classes operate on model

objects.

8. Design Decisions & Rationale

Java collections are used to manage multiple student records without an external database. A LinkedHashMap keeps student IDs unique while preserving insertion order. Streams are used for searching, filtering, sorting, and averaging. java.time.LocalDate records update dates and attendance dates. CSV was selected because it is simple, portable, and easy to inspect during evaluation. The application remains command-line based for direct terminal execution.

9. Implementation Details

The Student model uses encapsulated fields and methods for updating data. StudentService provides CRUD and search operations. AttendanceService stores AttendanceRecord objects and calculates percentages using stream operations. PerformanceService adds marks and creates reports. CsvStorage reads and writes persistent student summary data using java.nio.file.Files. InputValidator prevents empty fields, invalid email formats, and out-of-range numbers.

10. Screenshots / Results

The following screenshots are actual execution evidence captured while testing the completed application. They demonstrate the main application menu, attendance processing, reporting, and successful data saving.

11. Testing Approach

Manual testing covered student creation and viewing, attendance updates, mark entry, average and grade calculation, report generation, saving data, and restarting the application to confirm that saved student summary data was loaded. Additional validation scenarios include duplicate IDs, invalid email input, marks outside 0–100, attendance values where present exceeds total, and missing student IDs.

12. Challenges Faced

The main implementation challenges were separating responsibilities into packages, validating console input without external libraries, maintaining persistent CSV data, and calculating reports from collections. These were addressed with service classes, reusable validation methods, exception handling, try-with-resources, and Streams API operations.

13. Learnings & Key Takeaways

The project demonstrates practical application of OOP, encapsulation, packages, collections, exception handling, file handling, Streams API, java.time, input validation, and command-line application design. It also reinforces the importance of modular code and documentation.

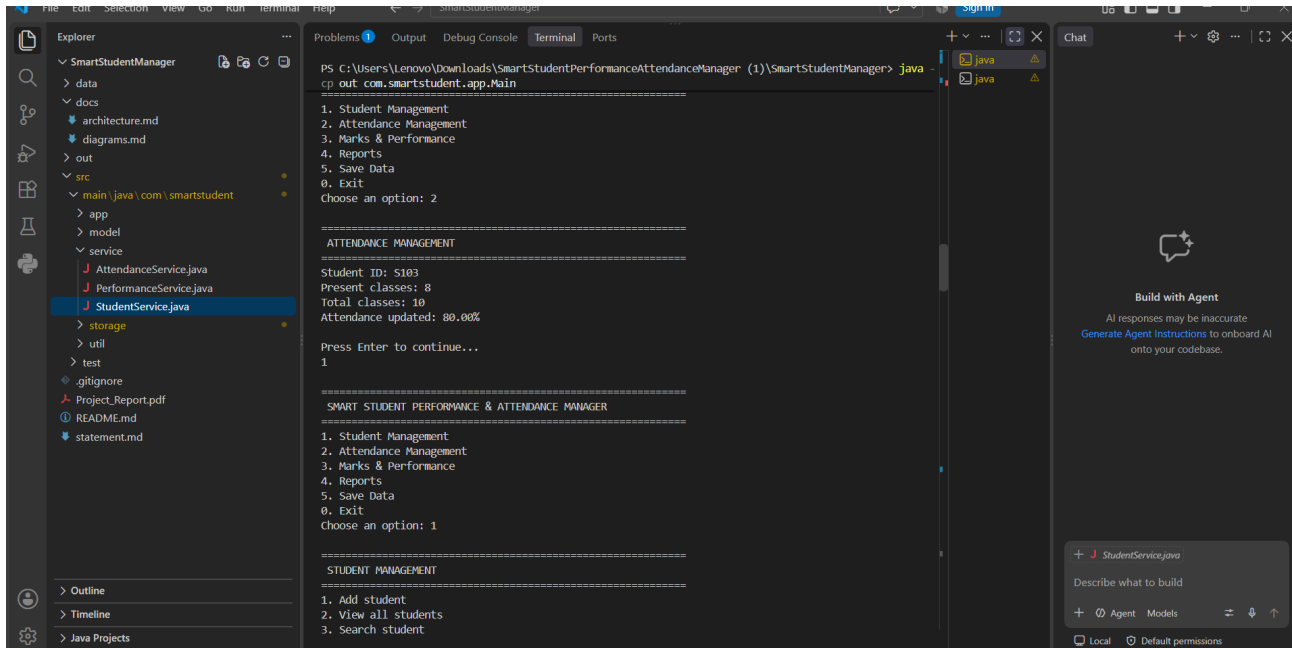
14. Future Enhancements

Possible future improvements include role-based login, a relational database, import/export for complete attendance and marks history, richer analytics, a GUI or web front end, unit testing with JUnit, and encrypted or authenticated storage.

15. References

The project requirements and submission structure were based on the VITyarthi Build Your Own Project instruction document supplied for the assignment. The implementation uses standard Java SE APIs available in Java 17+.

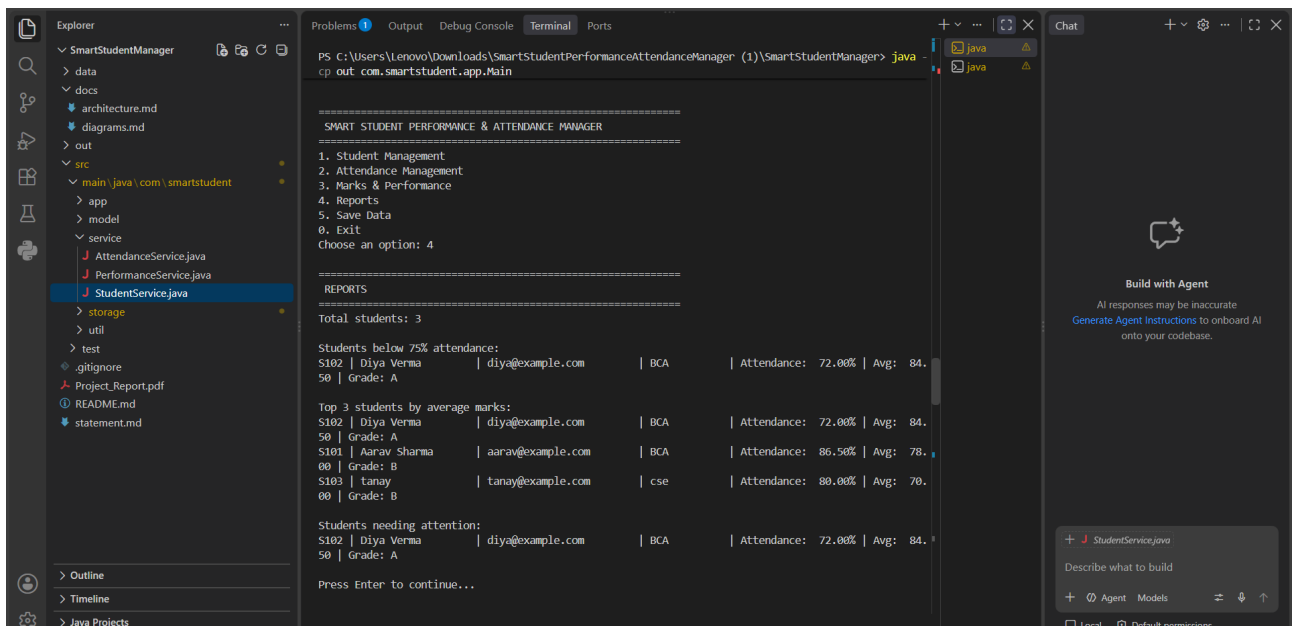
Project Execution Evidence



The screenshot shows the VS Code interface with the 'SmartStudentManager' project. The terminal window displays the output of the 'Attendance Management' test. The output shows the program running and displaying the following information:

```
PS C:\Users\Lenovo\Downloads\SmartStudentPerformanceAttendanceManager (1)\SmartStudentManager> java -cp out com.smartstudent.app.Main
=====
1. Student Management
2. Attendance Management
3. Marks & Performance
4. Reports
5. Save Data
0. Exit
Choose an option: 2
=====
ATTENDANCE MANAGEMENT
Student ID: S103
Present classes: 8
Total classes: 10
Attendance updated: 80.00%
Press Enter to continue...
1
=====
SMART STUDENT PERFORMANCE & ATTENDANCE MANAGER
=====
1. Student Management
2. Attendance Management
3. Marks & Performance
4. Reports
5. Save Data
0. Exit
Choose an option: 1
=====
STUDENT MANAGEMENT
=====
1. Add student
2. View all students
3. Search student
=====
```

Figure 1. Attendance Management test: S103 with 8 present classes out of 10 and 80.00% attendance.



The screenshot shows the VS Code interface with the 'SmartStudentManager' project. The terminal window displays the output of the 'Reports' module. The output shows the program running and displaying the following information:

```
PS C:\Users\Lenovo\Downloads\SmartStudentPerformanceAttendanceManager (1)\SmartStudentManager> java -cp out com.smartstudent.app.Main
=====
SMART STUDENT PERFORMANCE & ATTENDANCE MANAGER
=====
1. Student Management
2. Attendance Management
3. Marks & Performance
4. Reports
5. Save Data
0. Exit
Choose an option: 4
=====
REPORTS
Total students: 3
Students below 75% attendance:
S102 | Diya Verma | diya@example.com | BCA | Attendance: 72.00% | Avg: 84.50 | Grade: A
Top 3 students by average marks:
S102 | Diya Verma | diya@example.com | BCA | Attendance: 72.00% | Avg: 84.50 | Grade: A
S101 | Aarav Sharma | aarav@example.com | BCA | Attendance: 86.50% | Avg: 78.00 | Grade: B
S103 | Tanay | tanay@example.com | cse | Attendance: 80.00% | Avg: 70.00 | Grade: B
Students needing attention:
S102 | Diya Verma | diya@example.com | BCA | Attendance: 72.00% | Avg: 84.50 | Grade: A
Press Enter to continue...
```

Figure 2. Reports module showing total students, students below 75% attendance, top 3 students by average marks, and students needing attention.

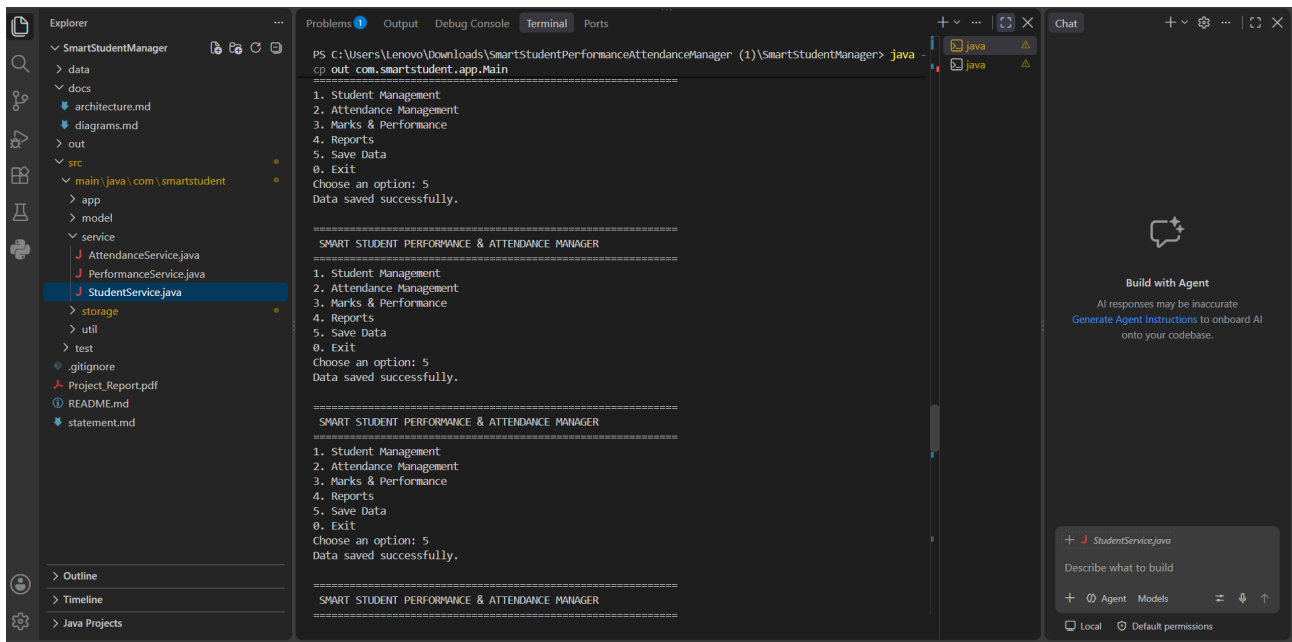


Figure 3. Save Data test showing successful persistence and return to the main application menu.

Requirements Traceability

Instruction area	Project evidence
3 major functional modules	Student Management; Attendance Management; Marks & Performance; Reports
4+ non-functional requirements	Usability; Reliability; Maintainability; Resource efficiency; Portability
5–10 meaningful classes/files	Modular Java classes across model, service, storage, util, app, plus test class
Package structure	model, service, storage, util, app
Testing	ProjectValidationTest plus manual execution tests
Project evidence	Actual terminal screenshots included in this report
Documentation	README.md, statement.md, architecture.md, diagrams.md

Submission note: Keep the source code, README.md, statement.md, docs/, data/, and this final report together in the project repository. Before final submission, verify that the repository contains the exact version of the code that was tested.